

Monolithic beam / rigid-body coupling: development plan

30 September 2026 · Colm Mc Alister · beamFoam `colm_6D0F_solver`, moorFV `feature-beamFoamCoupling-colm`

Companion to `rigidBodyCouplingPRSummary.pdf`, which describes what the code does today.

Progress: Phase 0 completed 30 September 2026 (beamFoam `a2dcb75`). Phase 1 completed 30 September 2026. Phase 2 completed 30 September 2026. Phase 3 completed 30 September 2026.

Aim. Solve the motion of a rigid body attached to a beam end inside beamFoam's BlockEigen system, so beam and body are advanced together by one Newton iteration, instead of taking turns between beamFoam and `sixDoFRigidBodyMotion`.

Approach. Write the body's equation of motion as a residual in force units, eliminate the attachment displacement using the pin compatibility condition, and let the body state come entirely from the converged solve. Build it in phases, starting with a fast test setup and translation only, and verify each phase against analytic cases and a converged partitioned reference.

1. Why the current code isn't coupled

1. **Staged mode leaves the body rows uncoupled.** The attachment target stays fixed at the last accepted body state, so the body block is an identity and the solve is the partitioned loop.
2. `fullyImplicit` **mixes variables.** The body rows are scaled to displacement units, and the right-hand side subtracts the *beam-end* displacement from the previous iterate. That value includes `initialW` and is not the body's centre-of-rotation displacement, so attachment-point and centre-of-rotation quantities get mixed. This probably explains its convergence problems.
3. **Velocity and acceleration come from the explicit force**, not from the solved displacement.
4. **moorFV's** `FvBeamNewmark` **recomputes the body state** after `restrain()`, discarding the solve.

Two facts from the code make the job smaller:

- **The attachment is a pin.** `Theta` has a zero-moment BC on the attachment patch, so only force passes between beam and body. That leaves 3 compatibility equations.
- **The beam equations are already incremental.** A `fixedValue` `W` boundary enters as `source == K_b · (W_b - W_b,prevIter)` with `K_b = CQW/pDelta`. That source term is what becomes a matrix column. `coupledTotalLagNewtonRaphsonBeamBoundaryConditions.C:339–420`

2. Target formulation

Each Newton iteration k solves for increments of all unknowns together.

Unknowns

beam: ΔW , $\Delta\theta$ in every cell body: Δx (CoR translation), $\Delta\phi$ (rotation)

Compatibility (eliminated, not solved for)

$$W_b = x + (R - I) \cdot r\theta + W_{b,0} \quad \rightarrow \quad \Delta W_b \approx \Delta x + \Delta\phi \times r$$

Beam rows

existing assembly; the attachment's $K_b \cdot \Delta W_b$ source term moves to the left-hand side as blocks on the Δx and $\Delta\phi$ columns

Body translation rows (force units)

$$R_x = m \cdot (x^k - \tilde{x}) / (\beta \cdot \Delta t^2) - F_{\text{fluid}} - F_{\text{beam}}(W^k, \theta^k, x^k)$$

$$\tilde{x} = x_n + \Delta t \cdot v_n + (\tfrac{1}{2} - \beta) \cdot \Delta t^2 \cdot a_n$$

$$\partial R_x / \partial x = m / (\beta \cdot \Delta t^2) \cdot I + K_b$$

$$\partial R_x / \partial W_{\text{cell}} = -K_b$$

$$\partial R_x / \partial \theta_{\text{cell}} = \text{from CQTheta (already computed, currently unused)}$$

Body rotation rows

$$R_\phi = \text{inertia term} - \tau_{\text{fluid}} - r \times F_{\text{beam}}$$

Jacobian: inertia term, $r \times K_b(\dots)$ terms, geometric term from the arm rotating

After convergence (same for rotation)

$$a = (x - \tilde{x}) / (\beta \cdot \Delta t^2) \quad v = v_n + \Delta t \cdot [(1 - \gamma) \cdot a_n + \gamma \cdot a]$$

→ complete body state handed to moorFV, which only stores it

- **Residual exact, Jacobian may be approximate.** The converged answer depends only on the residual. An approximate Jacobian slows convergence but doesn't change the result.
- **Set W_b from the current body iterate** every Newton iteration, not from the last accepted state.
- **Compute F_{beam} from the same discrete boundary flux the beam matrix uses**, so action and reaction match exactly.
- **Include the body's residual and increment in the convergence check.**

Design choices

Choice	Recommendation	Reason
Attachment constraint	Eliminate <code>W_b</code>	Keeps the matrix square ($6N + 6$) and fits the existing coefficient code. Lagrange multipliers would give the attachment force directly, but make a saddle-point system.
Units of body rows	Force	Same units as the beam rows. The coupling appears as <code>-K_b</code> in both off-diagonal blocks, which gives a symmetry check.
Who owns the body state	beamFoam during the step, moorFV stores it	A beamFoam-side body object also works in standalone beamFoam, which makes fast tests possible.
Time scheme	Same for beam and body	The beam currently uses <code>Euler</code> for <code>d2dt2</code> , the body Newmark ($\beta = 0.25$, $\gamma = 0.5$). Run the beam with Newmark for validation.
Rotation update	<code>Q = exp(ϕ) · Q_n</code> , inertia in global axes	Simple to linearise. It won't match moorFV's symplectic <code>rotate()</code> bit for bit, so validate by convergence.

3. Phases

Phase 0: a fast test setup completed

An `InterFoam` run takes about 6 minutes, which is too slow to debug with.

- A `rigidBodyEnd` object in `beamFoam` that owns mass, inertia, state and the Newmark update, usable from standalone `beamFoam` and from `moorFV`.
- Three beam-plus-point-mass cases with no fluid: static hanging mass, axial oscillation, small-angle pendulum.
- One switch, `rigidBodyCoupling partitioned | monolithic`, in place of the five current ones.

Pass: the partitioned standalone cases run in seconds and match the analytic results (hanging mass: end force = mg ; oscillation: frequency within expected accuracy; pendulum: small-angle period).

Completed successfully on 30 September 2026 (`beamFoam a2dcb75`). All 7 analytic checks in `tutorials/rigidBodyEnd/checkRigidBodyEnd.py` pass, and the whole set runs in about 25 s:

- Hanging mass: end force and static stretch within $1.4e-6$.
- Axial oscillation: period within $5e-8$ of the exact rod-plus-mass mode; mean and peak within $3e-4$.
- Pendulum: period within $1.2e-3$, peak angle within $9e-4$.
- `rigidBodyAndBeam_loop` and `_monolithic` reproduce their previous logs.

Notes: the body is a point mass (rotation is Phase 2). The legacy switches remain for the `moorFV` path; they go when Phase 1 replaces staged mode. The pendulum rod is clamped at the top because a rod pinned at both ends can twist freely. Two existing bugs were fixed: the Newmark predictor was re-applied on repeated solves within a step, and the gravity density check ran before `rho` was read. One coupling pass per step is unstable (amplitude +15% per period in the axial case), so the partitioned reference iterates to convergence.

Phase 1: translation only (body rotation constrained) completed

- Body rows in residual form, `w_b` eliminated, the attachment source term turned into matrix columns, and state recovery after convergence.
- A **finite-difference Jacobian check** behind a debug switch: perturb each body unknown and compare with the assembled column. This catches sign errors, which the Codex notes flag as unaudited.

Pass: near-quadratic Newton convergence; the analytic tests match; the result equals the partitioned scheme iterated to convergence. Both should converge to the same answer.

Completed successfully on 30 September 2026. `rigidBodyCoupling monolithic` solves the body's translation in the same Newton iteration as the beam. All 24 checks in `tutorials/rigidBodyEnd` pass:

- Analytic checks pass with monolithic coupling, to the same accuracy as partitioned.
- Jacobian: assembled body columns match finite differences to 1e-9 or better on all significant entries.
- Monolithic equals converged partitioned to within 3e-5 of peak values, time step by time step.
- Newton converges near-quadratically (e.g. residual $7.4e3 \rightarrow 1.5e-2 \rightarrow 1.3e-5$), with about a third of the beam Newton iterations of converged partitioned coupling (3.0 vs 8.9, 2.0 vs 6.1, 3.5 vs 9.3 per step).
- Stable with one solve per step, where one partitioned pass grows 15% per period.
- `rigidBodyAndBeam_loop` and `_monolithic` reproduce their previous logs.

How it works: the attachment `w` holds the current body iterate with a zero boundary increment, so the beam's `fixedValue` increment terms vanish and the body column carries them instead (`+CQW/pDelta` on the end cell's force rows, `+CMQW/pDelta` on its moment rows). The body row's dependence on the end cell and the attachment force come from assembling the boundary terms alone, so action and reaction match the beam's own discretisation. Body rotation rows stay identity (Phase 2). Found on the way: `explicitQ_` uses the strain from the last `updateSolutionVariables()`, not the current boundary value, which is why the standard `fixedValue` path adds the boundary increment explicitly.

Phase 2: rotation completed

- $\Delta\phi$ rows with the moment of inertia in global axes, the $\mathbf{r} \times \mathbf{F}$ coupling and the geometric term; orientation updated as $\mathbf{Q} = \exp(\phi) \cdot \mathbf{Q}_n$.

Pass: pendulum and tilted-arm cases match the converged partitioned reference; the finite-difference check passes for the rotation columns.

Completed successfully on 30 September 2026. The body now has a moment of inertia, a centre of mass away from the attachment, and optional linear and angular damping; its rotation is solved in the same Newton iteration as the beam. All 54 checks in `tutorials/rigidBodyEnd` pass, including three new rotating cases:

- `hangingBody` : the body swings 90° and settles with its centre of mass below the attachment (misalignment $1.6\text{e-}6$ rad, force within 0.4% of mg).
- `freeRotation` : torque-free spin conserves global angular momentum to $1\text{e-}6$ and energy to $3\text{e-}8$.
- `compoundPendulum` : both mode frequencies of the rod + body double pendulum match the analytic values ($8\text{e-}4$ and $4\text{e-}3$).
- Jacobian: rotation columns match finite differences to $1\text{e-}7$ on significant entries.
- Monolithic equals converged partitioned to within $1\text{e-}4$ of peak values ($5\text{e-}4$ for the compound pendulum), with 2.3–4.4× fewer beam Newton iterations.
- `rigidBodyAndBeam_loop` and `_monolithic` reproduce their previous logs.

How it works: the orientation is $\exp(\theta) \cdot \mathbf{Q}_n$ with θ the step rotation vector, integrated with the same Newmark parameters as translation. The attachment displacement is $\mathbf{x} + (\mathbf{R} - \mathbf{I}) \cdot \mathbf{r}_0$. The rotation rows are the moment balance about the centre of mass (inertia, gyroscopic, damping, $\mathbf{r} \times \mathbf{F}$), and the rotation columns include the geometric term and the $\text{SO}(3)$ left Jacobian of θ . Differences from the plan: rotation uses the global inertia $\mathbf{R} \cdot \mathbf{J} \cdot \mathbf{R}^T$ as planned, and damping was added so a large-rotation case can settle physically. Known small omission: the end cell's moment rows depend on the attachment displacement slightly beyond $\text{CMQW}/p\Delta$ (about $1\text{e-}9$ of the column). In `compoundPendulum` the rod's undamped axial mode pumps energy into the body mode in beats; the mode frequencies are unaffected.

Phase 3: moorFV integration completed

- A new `sixDoFFvBeamSolvers` type (e.g. `beamFoamCoupled`) that computes the fluid force, calls the restraint and accepts beamFoam's state as is: no Newmark recomputation, no acceleration relaxation on the beam's part.
- Pass `F_fluid` and `tau_fluid` explicitly in `RigidBodyStepData` instead of hiding them in `curr.acceleration`; add any other restraints to that external force.
- Settle the `restrain()` interface. It could return to `const` if the new solver owns the state update.

Pass: the body state written by moorFV equals the state beamFoam solved for.

Completed successfully on 30 September 2026. The moorFV sixDoF solver type `beamFoamCoupled` hands the fluid force and moment (with the weight) and any other restraints to the `finiteVolumeBeam` restraint as an applied load; beamFoam solves the beam and the body's translation and rotation in one Newton iteration, and moorFV copies the accepted state as it is (no Newmark recomputation, no acceleration relaxation).

- New interFoam case `tutorials/rigidBodyAndBeam_beamFoamCoupled` (the loop case with only the solver type changed): the body state moorFV stores equals beamFoam's solved state at all 25 write times, to about $2e-10$ (the history files' precision). Its trajectory stays within $2.5e-4$ of peak displacement of the loop case; run time 338 s against 351 s.
- `rigidBodyAndBeam_loop` and `_monolithic` reproduce their previous logs; all 54 standalone checks still pass.

How it works: new `beamModel` interface (`initialiseRigidBodyEnd`, `setRigidBodyEndExternalLoad`, `getRigidBodyEndState`); on the first call the restraint sets up beamFoam's body from the moorFV body (mass, inertia, orientation, centre of mass, velocities; Newmark and damping from the solver dictionary), with no gravity in beamFoam because moorFV's load already contains the weight. `rigidBodyEnd` gained an initial orientation, with inertia and arm in body axes. The `restrain()` interface is unchanged (the new solver does not use it for the beam). Limits: the centre of rotation must be the centre of mass; one `finiteVolumeBeam` restraint; no constraints; no restarts yet; other restraints and the fluid force are explicit. The Jacobian check now uses central differences; it shows one small omitted term in the interFoam case (end-cell moment rows against the attachment displacement, about $4e-5$ of the moment rows).

Phase 4: validation in interFoam

- `rigidBodyAndBeam`: monolithic against the loop with outer iterations converged.
- Stress test with a lighter body, stiffer line or larger Δt , where the loop needs many outer iterations or diverges. Report stability and cost.

Pass: close agreement on the base case; monolithic stable where the one-pass loop is not, at a competitive cost.

Phase 5: several mooring lines and parallel runs

- `mooringCase-H12T20` has 4 lines, each solved by its own restraint. Rather than one giant matrix, use **static condensation**: condense each line's tangent onto its attachment point (3×3) and add them all to the body rows.
- Check parallel behaviour; restraints currently run on the master process only.

Pass: the 4-line case runs monolithically and matches a converged partitioned run.

4. Limitation

The fluid force is still taken from the start of the step. This work removes the splitting error between beam and body, but not between fluid and body. For light bodies in water, added-mass instability can still require interFoam outer correctors, so Phase 4 should test with that in mind.